Enhancement One Narrative: Software Design and Engineering

The artifact for this enhancement is a 3D interactive scene built in C++ using OpenGL, created originally for CS 330: Computational Graphics and Visualization. The scene depicts a campfire environment, complete with a ground and water plane, three logs, three animated flame cones, a ring of ten stones arranged in a circle, and directional and point lighting. The surrounding architecture, including the main render loop, shader manager, and camera and view handling, was provided as instructor scaffolding by Professor Brian Battersby, while my own work is the scene content in PrepareScene() and RenderScene(), along with the keyboard-driven projection toggle in ProcessKeyboardEvents().

I selected this artifact for my ePortfolio because it demonstrates my ability to work inside an existing, partially unfamiliar codebase, understand its architecture well enough to extend it safely, and identify real defects in code I did not originally write. That last part is the core of this enhancement. During my code review, I audited SceneManager.cpp and found two genuine bugs, not stylistic nitpicks. The first was in DestroyGLTextures(), which was calling glGenTextures() instead of glDeleteTextures(). As written, the method that was supposed to free GPU texture memory on shutdown was instead generating new, unused texture handles and overwriting the stored IDs, meaning the original textures were never released and a new leaked handle was created every call. The second was in FindMaterial(), which had an unconditional return(true) outside its search loop. The loop correctly tracked whether a tag was found in a local bFound flag, but the function ignored that flag entirely and always reported success, even when no matching material existed. Any caller checking the return value to decide whether it was safe to use the material output parameter would have been misled by this.

I fixed both defects by making the code do what its own name and structure implied it should do. Swapping the GL function call in DestroyGLTextures(), and returning the tracked bFound value instead of a hardcoded true in FindMaterial(). I also refactored ProcessKeyboardEvents(), where the P and O key toggle logic was two nearly identical, copy-pasted blocks differing only in which key, flag, and target value they used. I extracted that

repeated pattern into a single reusable ToggleKeyState() helper method, cutting the duplicated logic from sixteen lines down to two calls.

For my final enhancement in this category, I completed the RenderScene() refactor from my original enhancement plan. Every object in the scene was drawn with its own hand-written block that set the scale, rotation, position, texture, and color, then drew the shape, which added up to roughly 250 lines of mostly repeated code. I created a SCENE_OBJECT struct that holds everything needed to draw one object, built a list of all the objects once in PrepareScene(), and replaced the repeated blocks in RenderScene() with a single loop. The objects use the same values as before, so the scene looks the same, but adding or moving an object now means changing one line instead of copying a whole block. This builds on the same reusable components outcome as the ToggleKeyState() refactor, and it taught me how much easier code is to work with when the data is kept separate from the logic that uses it.

This enhancement allowed me to meet the course outcomes I planned to address for this category back in Module One. It most directly demonstrates the outcome of using well-founded techniques, skills, and tools in computing practices to implement solutions that deliver value, since the DestroyGLTextures() and FindMaterial() fixes are great examples of applying correct, defensible engineering judgment to existing code rather than accepting it at face value. It also supports the outcome around developing a security mindset that anticipates design flaws, in a resource-management sense. An unbounded texture leak is exactly the kind of defect that would degrade a long-running application over time, and fixing it reflects an awareness of where resources were not being managed safely. The ToggleKeyState() refactor supports the outcome around using reusable components and reducing unnecessary complexity in design. I do not have any updates to my outcome-coverage plan at this stage. The enhancement landed where I expected it to when I planned it during the code review.

Reflecting on the process, the biggest lesson was how much can be learned about code quality simply by reading carefully and checking that a method's implementation actually matches its name and its documented purpose. DestroyGLTextures() had a comment stating that

it frees memory, and it compiled without errors, but it did the opposite of what it claimed. That is a good reminder that a codebase can look clean and well-organized at a glance while still hiding a real bug, and that reviewing someone else's code, including instructor-authored code, means verifying behavior rather than simply skimming for structure. The main challenge was not the fixes themselves, which were small once identified, but the work of reading through unfamiliar methods like FindMaterial() and SetShaderMaterial() closely enough to notice that a return value was being computed correctly in one place and then thrown away in another. I also had to think carefully about the ToggleKeyState() refactor to make sure a single helper could correctly generalize two toggles that set the same target variable to opposite values, without losing the per-key debounce behavior each one needed.